

- 랭체인을 활용한 에이전트 개발

↳ GPT API 2 개발할 때와 랭체인을 활용할 때의 차이점을 이해하고, 다양한 문서 기반 작업을 할 수 있게 하는 RAG나 인터넷 검색, PDF 사용 등의 기능을 손쉽게 추가하는 방법을 알아보자.

08-1. 랭체인으로 챗봇 만들기.

- 랭체인이란?

· 랭체인 Langchain은 언어 모델에 기반한 AI 애플리케이션을 쉽게 개발할 수 있도록 도와주는 프레임워크이다.

- 요약, 검색, 문서 생성, 질의응답 등 여러 기능을 손쉽게 구현할 수 있다.
- 다른 언어 모델을 쉽게 교체할 수 있다.

[실습] 랭체인과 오픈 AI의 GPT AI 비교하기.

1. 랭체인과 랭체인에서 오픈 AI 모델을 사용할 수 있게 해주는 langchain-openai 설치.

> pip install langchain langchain-openai

2. 주피터 노트북에서 실행

① 라이브러리 준비, 모델 선언

```
> from langchain_openai import ChatOpenAI  
model = ChatOpenAI(model="gpt-4o-mini")
```

② 사용자의 메시지를 의미하는 HumanMessage를 불러오고, model.invoke에 메시지를 리드 형식으로 전달한다.

* 오픈 AI의 API에서 다음과 같은 형태로 메시지 전달

ex) {"role": "user", "content": " ————— "}

↳ langchain에서는 invoke에 HumanMessage를 담아서 전달

ex)

```
from langchain_core.messages import HumanMessage
```

```
model.invoke([HumanMessage(content=" ————— ")])
```

↳ langchain을 사용한 경우는 AI Message 클래스 객체 안에 content가 담긴 형태로 반환된다.

[실습] 랭체인으로 멀티턴 대화하기

- 대화 내용 리스트에 메시지 추가하기

↳ 이전 AI API 스타일의 멀티턴 코드를 수정하자.

* 기존 코드와 비교할 수 있도록 오픈 AI API 스타일의 코드를
주석처리해서 남겨두자.

- 변경 사항

① 사용할 모델 정의 - 랭체인에서 제공하는 ChatOpenAI를
사용하므로 이전에 사용한 클라이언트 선언 코드는 필요하지 않다.

② messages 딕셔너리 형태 → SystemMessage로 전달

③ invoke로 받은 AI-response를 그대로 메시지 리스트에
추가

- 랭체인만의 메시지 히스토리

↳ AI API에서 멀티턴 대화를 위해 매번 사용자의 대화 내용을
추가하는 코드를 작성해야 했다.

랭체인만의 메시지 히스토리 기능을 사용하면 멀티턴 대화를 더 쉽게
구현할 수 있다

1. 메시지 히스토리를 단계별로 이해하기 위해 주피터 노트북 파일을 생성한다.

↳ 코드 작성

① 준비

· 필요 라이브러리

- InMemory Chat Message History - 메모리 내에서 메시지를
리스트 형태로 보관. 애플리케이션을 종료하면 내용이 사라짐으로
기록 사용하고 싶다면 파일이나 DB에 저장 필요

- Runnable With Message History - 모델을 생성할 때, 대화 기록을
함께 전달할 수 있도록 하는 클래스

- Chat OpenAI, Human Message

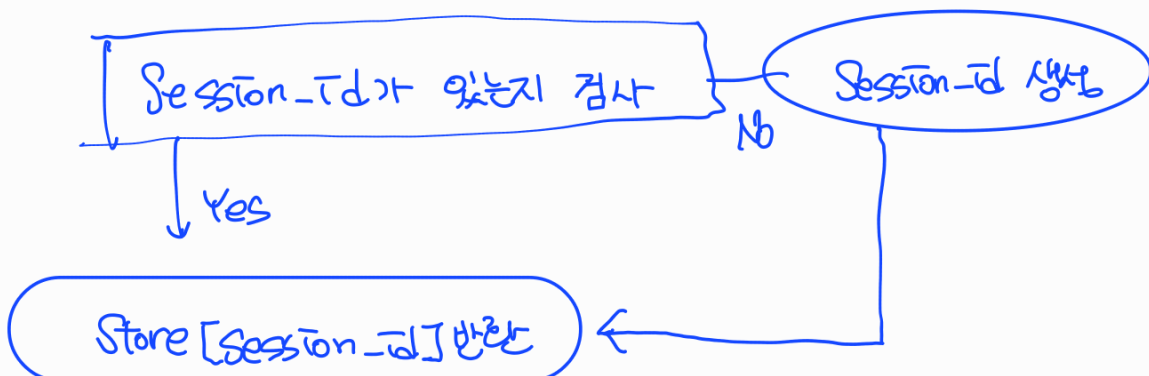
· model 선언

· Store 디렉터리 선언 - Session-Id를 기준으로 사용자의 대화 기록을
저장하는 디렉터리

② ^{함수 정의} get_session_history (세션 ID에 따라 대화 기록을 가져오는 함수)

↳ 매개변수: Session-Id: str → Base Chat Message History

↳ 매개변수를 객체로 가져온다.



2. 대화 시작하기 - ChatOpenAI를 바로 사용하는 model.invoke 대신 with_message_history()를 실행하고, session_id를 설정한 후 시작해보자.

```
> Config = {"Configurable": {"session_id": "abc2"}}
```

↳ 세션 ID를 설정하는 Config 객체 생성

```
response = with_message_history.invoke([  
    HumanMessage(content="안녕? 난 ㅇㅇㅇㅇㅇ."),  
    Config=Config,  
)
```

```
print(response.content)
```

3. 이어서 대화 내용을 잘 기억했는지 확인하기 위해 이름을 물어보자.

* session_id를 새로 입력하면, 이전 대화는 기억하지 못한다.

[실습] 스트림 방식으로 출력하기.

↳ .invoke → .stream 으로만 바꿔 주면 됨

08-2. LCEL로 체인 만들기

- LCEL (LangChain Expression Language)는 랭체인에서 복잡한 작업 흐름을 간단하게 만들고 관리할 수 있도록 돕는 도구이다.
- 랭체인에서는 이런 작업 흐름을 연결하는 것을 체인이라고 한다.
- LCEL을 이용하면 여러 줄로 표현해야 하는 작업 단계를 읽기 쉽게 작성할 수 있으며, 스트림 출력 등 여러 작업을 병렬로 처리할 수 있다.

[실습] 출력 파서인 체인

- 랭체인 기본 문법으로 챗봇 코드를 작성해보자. 랭체인 코드로 역할을 부여한 챗봇을 만들어보자. 단계별 학습을 위해 주피터 노트북 환경에서 실행하자.
- 1. GPT-4o-Mini 모델을 사용하여 시스템 프롬프트에 '미녀타마수'의 미녀(별) 역할을 맡아 그 캐릭터에 맞게 대답해 달라고 요청한다. 그리고 나를 비원 역할인 개스통이라고 소개하며 지극 신사를 제안한다.

↳ 도구명해보기

```
> from langchain_openai import ChatOpenAI  
model = ChatOpenAI(model="gpt-4o-mini")
```

```
from langchain_core.messages import HumanMessage,  
SystemMessage
```

```
messages = [
```

```
    SystemMessage (Content="너는 디어한 아수미 나쁜 미녀야.  
    1 캐릭터에 맞게 사용자 대화하라."),
```

```
    Human Message (Content="안녕? 저는 개드론입니다. 오늘  
    시간 관광이면 저녁 같이 먹을까요?"),
```

```
]
```

```
model.invoke(messages)
```

↳ 출력된 결과를 보면 AI Message 객체 안에 여러 정보가 있다.

만약, 텍스트 결과만 필요하다면 Str Output Parser를 사용하면 된다.

StrOutputParser는 런타임에서 제공하는 다양한 출력 포서 중 하나로,
텍스트만 추출해서 반환한다.

2. StrOutput Parse 3 반환 형식 설정하기.

① StrOutput Parse 라이브러리 임포트

② Parser 변수 선언

③ 답변 Parser 3 출력

3. LCEL의 체인을 이용해 1 연산자로 한 줄로 간단하게 연결하기.

```
Chain = model | parser
```

Chain. Invoke (messages)

[실습] 프롬프트 템플릿 이용하기.

- 다른 캐릭터로 변경해서 별미게 제안한다면 매번 messages의 내용을 바꿔서 테스트할 수도 있지만, **강제인의 프롬프트 템플릿**을 이용하면 필요한 부분만 수정하여 실행할 수 있다.

1. 프롬프트 템플릿 이용하기.

↳ 대화 중 변수를 만들고 싶은 부분을 중괄호로 감싸고 변수명을 지정한다.

① 준비

↳ ChatPromptTemplate 라이브러리 import

↳ system-template, human-template 작성

② 프롬프트 템플릿 적용

↳ ChatPromptTemplate를 사용하여 'system', 'user' 지정

↳ prompt-template.invoke (↑ 안에 각 변수를 대입 실행)

③ 결과 출력

2. !연산자 이용해 체인 구성하기.

↳ 프롬프트 템플릿, 모델, 파서를 Chain으로 만들고, invoke 연산

08-3. 랭체인 도구를 에이전트 만들기.

- 랭체인 도구를 활용하면 직접 만든 함수나 다른 사람이 만든 기능을 챗봇에 쉽게 추가할 수 있다.

[실습] ① tool 디코레이터로 랭체인에 함수 연결하기.

- 이전에 API로 펄션 쿼리를 사용해 만들었던 시간을 알려주는 챗봇을 랭체인으로 개발해보자. ① tool 디코레이터를 사용하면 함수를 도구에 변환할 수 있다.

1. 주피터 노트북을 생성하고, 랭체인 스타일로 인터 보드를 설치하고 `invoke()` 메서드를 사용해보자.

2. 오픈 AI API의 펄션 쿼리를 받을 때 만들었던 `get-current-time` 함수를 가져와서 활용해보자.

API 펄션 쿼리 <code>get-current-time</code>	랭체인 <code>get-current-time</code>
매개변수: <code>timezone</code>	<code>time zone, location</code> ↳ 답변을 생성하기 위해서 주어
딕셔너리로 정보작성	"" "" 주석 처리로 문자열 문자열

3. 사용할 수 있는 도구를 리드와 디렉터리 형태로 만들고,

`.bind_tools()` 메서드를 사용하여 기존에 선언한 언어 모델에
도구를 등록할 수 있다.

4. `messages` 만들고 `tools` 를 사용해 답변을 생성하라.

1) Messages 설정

① System Message: "나는 사용자의 질문에 답변을 하기 위해 `tools` 를
사용할 수 있다."

Human Message: "무슨 지름 막대야?"

② `response = llm_with_tools.invoke(messages)`

↳ 정확하게 Content 에는 값이 없고, `tool_calls` 에는 도구 호출
필요한 정보가 포함되어 있다. 이번에는 AI가 함수 하나만 실행하면
된다고 판단 했지만, 2개를 실행해야 할 때는 for 문을 사용해야 한다.

5. for 문 사용하여 함수 호출 정보를 가져와 반복하기.

↳ 반복문 안 연산

① `tool_dict` 를 사용해 도 함수 선택

↳ `Selected_tool = tool_dict[tool_call['name']]`

② 도 호출 시 전달된 인자 출력

↳ `Print(tool-call["args"])`

③ 도 호출을 호출해 결과 반환

↳ `tool-msg = selected-tool.invoke(tool-call)`

6. 현재까지 대화한 내용이 담긴 messages를 `llm-with-tools.invoke`에 넣으면 답변이 나온다.

[실습] 파이단틱 사용하기.

• 파이단틱 Pydantic은 입력된 데이터의 유효성과 형식을 검증하고 특정 데이터 형식으로 명확하게 표현할 때 사용하는 라이브러리이다.

이전에 만든 주가 정보 `get_yh-stock-history` 함수

입력값 형식을 명확하게 정의하고, 런타임 오류를 예방하여

답변을 생성해보자.

1. Stock History Input 클래스를 만들어서 입력받을 변수의 데이터 형식과 설명을 작성한다.

↳ 1) 준비

① Pydantic BaseModel의 Field 라이브러리 임포트

② 클래스 생성

```
ticker: str = Field(..., title="주식 코드",  
description="주식 코드 (예: AAPL)")
```

```
Period: str = Field(..., title="기간", description="주식 데이터 조회 기간 (예: 1d, 1mo, 1y)")
```

2) 함수 수정 - get_hf_stock_history

① @tool 디코레이터 적용

② 매개변수를 클래스 형식으로 받도록 수정

3) message 수정

① HumanMessage 추가

② llm_with_tools.invoke 사용하여 답변 요청

③ 결과를 출력하고 message에 추가

4) 함수를 실행하고 결과를 메시지로 추가할 수 있도록 코드 추가.

↳ response.tool_calls로 함수를 실행하여 모든 실행하고,
그 결과를 messages에 추가.

5) 함수 실행 결과로 제공된 message를 llm_with_tools.invoke
다시 처리하기.

Of_4. 스트림 방식으로 출력하기.

- 애플리케이션의 도구를 이용할 때는 GPT의 응답이 단순히 텍스트 형태가
아니라 어떤 함수에 어떤 인자를 넣어 실행할지를 판단해서
목적까지 형태로 넘겨준다. 따라서 애플리케이션으로 별도 처리를 해야한다.

[실습] 도구 사용할 때, 스트림 출력하기

- 먼저 모델만 있을 때, 스트림 방식으로 출력하기.

↳ Invoke() → stream()을 사용

(각 단어가 어떻게 출력되는지 확인할 수 있도록
end='!' 추가)

- 도구를 추가했을 때, 스트림 출력

1. 도구 호출 결과를 순차로 받아와 최종 결과를 생성해준다.

1) messages

├── SystemMessage
└── HumanMessage

2) response Stream으로 선언

3) is-first = True

반복문으로 chunk 구문해서 이어붙이기

4) messages에 추가.

2. gathered은 AIMessageChunk 타입이므로 messages에 추가하여
체인에서 바로 활용할 수 있다.

함수 실행 과정은 스트림 방식이 필요하면 invoke로 실행하고,
[이 message에 추가하고 출력한다.

3. llm_with_tools.stream() 사용하면 스트림 방식으로 출력된다.

08-5. 스트림릿에 구현하기.

[실습] 체인 메모리에 기반한 멀티턴 챗봇 만들기.

1. streamlit과 체인에서 필요한 패키지들 импорт

1) streamlit

2) ChatOpenAI

3) InMemoryChatMessageHistory → 메모리에 대화기록 저장

4) RunnableWithMessageHistory → 메시지 기록 활용하 실행할 수 있는 wrapper

5) HumanMessage, AIMessage, SystemMessage

2. 제목 입력 - St. title ()

3. 스트림릿이 세션동안 유지하는 상태 저장용 딕셔너리 만들기.

↳ st. session_state (이러 messages 가 많다면 시스템 메시지로 처리)

4. 세션별 대화 히스토리를 담당하는 Store 생성 - {}

5. get_session_history 함수 선언

↳ session_id를 기준으로 고유한 대화 이력을 추적

1) session_id를 매개변수로 받는다

2) session_id에 대응하는 대화 이력 객체를 가져온다

3) 만약, session_id가 없다면 새로 생성해서 저장한다.

6. 런타임의 ChatOpenAI를 이용해 llm 변수 선언

↳ 언어모델을 선택하고 > RunnableWithMessageHistory 사용

7. for 반복문으로 각 메시지 내용을 St.Chat_message로 출력.

↳ St.Chat_message("user") 나 St.Chat_message("assistant")
를 사용해 채팅 인터페이스처럼 출력 가능

8. 드림릿 메시 코드 가져와서, 맞게 수정

9. 스트림 방식으로 출력

1) with_message_history.invoke() → .stream() 으로 수정

2) ai_response_bucket을 None 으로 선언하고, for를 사용하여
계속 이기 붙인다

[실습] 랭체인 메모리 없이 챗봇 만들기

- 대화 내용을 DB에 저장하거나 히스토리를 수정해야 할 때는
히스토리를 사용하지 않고 리스트로 직접 관리하는 것이 더 효율적일 수 있다.

1. 이전에 실행했던 코드를 가져와서 수정.

1) 오픈 AI의 API 기반 → 랭체인 기반

2) 드림릿에서 사용할 수 있도록 수정.

[실습] 도구 추가하고 스트림 방식으로 출력하기.

· 현재 시계를 알려주는 도구를 추가하고 결과를 스트림 방식으로 출력해보기.

1. get-current-time 함수를 가져와서 llm-with-tools로 정의한다

2. 스트림잇에서 도구 호출을 포함한 답변을 스트림 방식으로 출력.

3. 기존 메시지들의 ToolMessage도 스트림 방식으로 수정.